

SOLSTREAK

Lộ trình kỹ thuật xây dựng sản phẩm, dùng nội bộ cho team phát triển

Ba cửa kỹ thuật chính: Privy, Jupiter Earn, chương trình Anchor khóa kỳ hạn

UniHackfest 2026

Điểm cần xác nhận ngay trước khi lên lịch test

Có một mâu thuẫn giữa hai nguồn tài liệu cần đội tự tay xác nhận trước khi chốt kế hoạch test, để không mất thời gian sửa lại giữa chừng:

- Tài liệu CPI chính thức của Jupiter liệt kê địa chỉ chương trình Lending và Liquidity trên cả Mainnet lẫn Devnet, cho thấy chương trình có tồn tại trên Devnet.
- Tuy nhiên, ví dụ chính thức của gói SDK tiện ích (Earn SDK) chỉ dùng tham số cluster với giá trị mainnet, chưa có ví dụ nào dùng devnet. Rất có thể chương trình có mặt trên Devnet nhưng không có thanh khoản hoặc trạng thái hoạt động, nên gọi qua SDK sẽ thất bại dù địa chỉ chương trình tồn tại.

Việc cần làm trong ngày đầu tiên: thử gọi thẳng `getDepositLx` với cluster devnet trên một ví test, xem có lỗi hay không. Nếu lỗi, chuyển sang phương án dùng local validator giả lập lại trạng thái Mainnet để test, kèm một khoản tiền thật rất nhỏ khi cần kiểm chứng lần cuối trước demo.

Luồng ký giao dịch tổng thể

Toàn bộ sản phẩm sống chết ở một luồng duy nhất: build instruction từ Jupiter, ký bằng ví Privy, gửi lên mạng. Đây là luồng bắt buộc phải chạy được trước khi làm bất cứ tính năng nào khác.

```
User bam "Nap 10 USDC"
Frontend gọi getDepositIx(amount, USDC, userPubkey) từ Jupiter SDK
Jupiter trả về instructions (chưa ký)
Frontend build VersionedTransaction kèm blockhash mới nhất
Frontend gọi Privy: signTransaction(tx)
Privy trả về giao dịch đã ký (khoa riêng không rơi khỏi thiết bị quản lý của Privy)
Frontend gửi giao dịch qua sendRawTransaction lên Solana Mainnet
Mạng trả về signature
Frontend báo signature cho Backend (coi là thành công tạm thời)
Backend tự xác minh lại giao dịch thật sự trên chain, rồi mới cập nhật chuỗi ngay
```

Giai đoạn 0: quyết định cần chốt trước khi viết dòng code đầu tiên

Vấn đề	Quyết định khuyến nghị
Jupiter Earn không rõ có Devnet dùng được hay không	Thử gọi SDK với devnet trước, nếu lỗi thì test bằng local validator giả lập trạng thái Mainnet, kiểm chứng lần cuối bằng tiền thật số rất nhỏ
Dùng giao dịch kiểu cũ hay VersionedTransaction	Dùng VersionedTransaction ngay từ đầu, đúng chuẩn Jupiter đang dùng, Privy hỗ trợ cả hai kiểu
Ai trả phí gas lúc demo	Giai đoạn demo để chính người dùng trả, đơn giản hơn nhiều. Việc nền tảng tài trợ phí gas để sau, vì thêm một chữ ký phía backend dễ làm luồng phức tạp và dễ lỗi hơn
Lãi cố định cam kết theo tier, trong khi Jupiter trả lãi thả nổi	Đây là rủi ro mô hình lớn nhất: nếu cam kết 4%/năm cho kỳ hạn 12 tháng nhưng Jupiter chỉ còn trả 3%, nền tảng lỗ phần chênh lệch. Ở bản demo, hiển thị rõ đây là lãi ước tính, có thể biến động theo thị trường, không cam kết tuyệt đối
Chọn stack kỹ thuật	Next.js cho frontend, Postgres (ví dụ Supabase) cho phần lưu trạng thái chuỗi ngày, một cron job định kỳ. Phần backend nên giữ càng gọn càng tốt

Giai đoạn 1: tích hợp Privy (tuần 1)

Cài đặt

```
npm install @privy-io/react-auth @solana/web3.js bn.js
```

Cấu hình provider và hook ví

```
// app/providers.tsx
'use client';
import { PrivyProvider } from '@privy-io/react-auth';

export function Providers({ children }: { children: React.ReactNode }) {
  return (
    <PrivyProvider
      appId={process.env.NEXT_PUBLIC_PRIVY_APP_ID!}
      config={{
        loginMethods: ['email', 'sms'],
        embeddedWallets: {
          solana: { createOnLogin: 'users-without-wallets' },
        },
        appearance: { walletChainType: 'solana-only' },
      }}
    >
      {children}
    </PrivyProvider>
  );
}

// hooks/useSolStreakWallet.ts
'use client';
import { usePrivy } from '@privy-io/react-auth';
import { useSolanaWallets } from '@privy-io/react-auth/solana';

export function useSolStreakWallet() {
  const { ready, authenticated, login, logout, user } = usePrivy();
  const { wallets } = useSolanaWallets();
  const embedded = wallets.find(w => w.walletClientType === 'privy');
  return { ready, authenticated, login, logout, user, wallet: embedded };
}
```

Vấn đề sẽ gặp ở giai đoạn này

- Luôn kiểm tra đủ điều kiện ready, authenticated và ví tồn tại trước khi thao tác, tránh gọi hàm khi ví chưa sẵn sàng trong lúc đang tải.
- Tên hook và API của Privy có thể thay đổi giữa các phiên bản, nên ghim đúng phiên bản trong package.json và đối chiếu tài liệu đúng phiên bản đó, không copy code từ bài viết cũ.
- Cần lưu lại việc gắn user_id với wallet_address ở backend, qua webhook khi tài khoản được tạo hoặc gọi API lần đầu người dùng đăng nhập. Tuyệt đối không lưu bất kỳ thông tin nào liên quan đến khóa riêng.

Xem như hoàn thành giai đoạn này khi: đăng nhập bằng email nhận mã xác thực, thấy được địa chỉ ví Solana hiện trên màn hình, và ký thử được một tin nhắn kiểm tra.

Giai đoạn 2: tích hợp Jupiter Earn (tuần 1-2)

Lớp dịch vụ nạp và rút

```
npm install @jup-ag/lend

// lib/jupiter.ts
import {
  Connection, PublicKey, TransactionInstruction,
  TransactionMessage, VersionedTransaction,
} from '@solana/web3.js';
import { getDepositIxs, getWithdrawIxs } from '@jup-ag/lend/earn';
import BN from 'bn.js';

export const USDC_MINT = new PublicKey('EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v');
const connection = new Connection(process.env.NEXT_PUBLIC_RPC_URL!, 'confirmed');

function toTxIx(raw: any): TransactionInstruction {
  return new TransactionInstruction({
    programId: new PublicKey(raw.programId),
    keys: raw.keys.map((k: any) => ({
      pubkey: new PublicKey(k.pubkey),
      isSigner: k.isSigner,
      isWritable: k.isWritable,
    })),
    data: Buffer.from(raw.data),
  });
}

export async function buildDepositTx(userPubkey: PublicKey, amountUsdc: number) {
  const { ixes } = await getDepositIxs({
    amount: new BN(Math.round(amountUsdc * 1_000_000)),
    asset: USDC_MINT,
    signer: userPubkey,
    connection,
    cluster: 'mainnet',
  });

  const { blockhash } = await connection.getLatestBlockhash('confirmed');
  const msg = new TransactionMessage({
    payerKey: userPubkey,
    recentBlockhash: blockhash,
    instructions: ixes.map(toTxIx),
  }).compileToV0Message();

  return new VersionedTransaction(msg);
}

export async function buildWithdrawTx(userPubkey: PublicKey, amountUsdc: number) {
  const { ixes } = await getWithdrawIxs({
    amount: new BN(Math.round(amountUsdc * 1_000_000)),
    asset: USDC_MINT,
    signer: userPubkey,
    connection,
    cluster: 'mainnet',
  });

  const { blockhash } = await connection.getLatestBlockhash('confirmed');
  const msg = new TransactionMessage({
    payerKey: userPubkey,
    recentBlockhash: blockhash,
    instructions: ixes.map(toTxIx),
  }).compileToV0Message();
  return new VersionedTransaction(msg);
}
```

Nối với ví Privy để ký và gửi giao dịch

```
// components/DepositButton.tsx
'use client';
```

```

import { Connection, PublicKey } from '@solana/web3.js';
import { buildDepositTx } from '@lib/jupiter';
import { useSolStreakWallet } from '@hooks/useSolanaWallet';

const connection = new Connection(process.env.NEXT_PUBLIC_RPC_URL!, 'confirmed');

export function DepositButton({ amount }: { amount: number }) {
  const { wallet } = useSolStreakWallet();

  async function onDeposit() {
    if (!wallet) return;
    const userPk = new PublicKey(wallet.address);

    const tx = await buildDepositTx(userPk, amount);
    const signedTx = await wallet.signTransaction(tx);
    const sig = await connection.sendRawTransaction(signedTx.serialize(), {
      skipPreflight: false, maxRetries: 3,
    });
    const confirmation = await connection.confirmTransaction(sig, 'confirmed');
    if (confirmation.value.err) throw new Error('Giao dịch thất bại on-chain');

    await fetch('/api/streak/report', {
      method: 'POST',
      body: JSON.stringify({ signature: sig, wallet: wallet.address }),
    });
  }

  return <button onClick={onDeposit}>Nap {amount} USDC</button>;
}

```

Việc đọc số dư và lãi tích lũy cho dashboard dùng các hàm đọc có sẵn của SDK (thông tin token, vị thế người dùng) hoặc gọi REST API của Earn, hiển thị chênh lệch giữa tiền gốc đã nạp (lưu ở cơ sở dữ liệu riêng) và giá trị hiện tại (lấy từ Jupiter) chính là phần lãi.

Những vấn đề chắc chắn sẽ gặp

Vấn đề	Triệu chứng	Cách xử lý
Blockhash hết hạn	Lỗi giao dịch hết hạn khi người dùng xác nhận chậm	Lấy blockhash ngay trước khi ký, không phải lúc build; nếu quá khoảng 60 giây thì dừng lại giao dịch
Ví mới chưa có tài khoản token	Lần nạp đầu tiên thất bại vì thiếu tài khoản token liên kết	SDK thường tự thêm sẵn hướng dẫn tạo tài khoản, nhưng vẫn cần test riêng trường hợp ví hoàn toàn mới
Ví không có SOL để trả phí	Người dùng nạp USDC nhưng ví không có SOL nên không ký nổi giao dịch	Cấp một khoản SOL rất nhỏ cho ví mới trong giai đoạn demo, xem là chấp nhận được cho mục đích trình diễn
RPC công cộng giới hạn tốc độ	Lỗi ngẫu nhiên khi gọi liên tục	Dùng nhà cung cấp RPC có hạn mức miễn phí ổn định hơn RPC công cộng mặc định
Nhầm đơn vị thập phân	Nạp 10 USDC nhưng hệ thống hiểu thành một số cực nhỏ	USDC có 6 chữ số thập phân, luôn quy đổi qua một hàm dùng chung duy nhất, có viết kiểm thử riêng cho hàm này

Xem như hoàn thành giai đoạn này khi: nạp thành công một khoản USDC thật, thấy được vị thế trên Jupiter, và rút thành công về ví, cả hai chiều đều thao tác qua ví Privy.

Giai đoạn 3: backend tính chuỗi ngày (tuần 2, chạy song song)

Cấu trúc dữ liệu tối thiểu

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  privy_user_id TEXT UNIQUE NOT NULL,  
  wallet_address TEXT UNIQUE NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT now()  
);  
  
CREATE TABLE deposits (  
  signature TEXT PRIMARY KEY,  
  wallet_address TEXT REFERENCES users(wallet_address),  
  amount_usdc NUMERIC NOT NULL,  
  block_time TIMESTAMPTZ NOT NULL,  
  verified BOOLEAN DEFAULT false  
);  
  
CREATE TABLE streaks (  
  wallet_address TEXT PRIMARY KEY,  
  current_streak INT DEFAULT 0,  
  longest_streak INT DEFAULT 0,  
  last_counted_day DATE  
);
```

Quy tắc tính chuỗi có vùng đệm

Một ngày chuỗi được tính theo giờ Việt Nam. Giao dịch nộp trước nửa đêm nhưng xác nhận trễ đến khoảng ba giờ sáng hôm sau vẫn được tính cho ngày cũ, để tránh mất chuỗi oan vì mạng nghẽn.

```
// lib/streak.ts  
import { addHours } from 'date-fns';  
import { toZonedTime } from 'date-fns-tz';  
  
const TZ = 'Asia/Ho_Chi_Minh';  
const GRACE_HOURS = 3;  
  
export function streakDayOf(blockTime: Date): string {  
  const adjusted = addHours(blockTime, -GRACE_HOURS);  
  return toZonedTime(adjusted, TZ).toISOString().slice(0, 10);  
}  
  
export function updateStreak(  
  prev: { current: number; lastDay: string | null },  
  newDay: string,  
) : { current: number; lastDay: string } {  
  if (prev.lastDay === newDay) return { current: prev.current, lastDay: newDay };  
  const yesterday = new Date(new Date(newDay).getTime() - 86_400_000)  
    .toISOString().slice(0, 10);  
  const current = prev.lastDay === yesterday ? prev.current + 1 : 1;  
  return { current, lastDay: newDay };  
}
```

Luôn xác minh lại trên chain, không tin dữ liệu phía client

```
// api/streak/report - backend tu xác minh chu kỳ giao dịch client gửi lên  
import { Connection } from '@solana/web3.js';  
  
export async function verifyDeposit(signature: string, expectedWallet: string) {  
  const connection = new Connection(process.env.RPC_URL!);  
  const tx = await connection.getTransaction(signature, {  
    maxSupportedTransactionVersion: 0,  
  });  
  if (!tx || tx.meta?.err) return null;  
  
  const keys = tx.transaction.message.staticAccountKeys.map(k => k.toBase58());  
  if (keys[0] !== expectedWallet) return null;
```

```
    return { blockTime: new Date((tx.blockTime ?? 0) * 1000), signature };  
  }
```

Nên chạy thêm một cron mỗi giờ, quét lịch sử giao dịch của các ví đã đăng ký để bắt những giao dịch mà phía client không kịp báo về, ví dụ khi người dùng đóng tab giữa chừng.

Vấn đề cần lưu ý: luôn xác minh đúng người ký là chủ ví đã đăng ký, tránh trường hợp báo cáo chữ ký của người khác; nạp hai lần trong cùng một ngày chỉ tính một lần theo đúng ngày chuỗi; và để chữ ký giao dịch làm khóa chính của bảng để việc chạy trùng lặp tự động bị bỏ qua.

Giai đoạn 4: vòng quay huy hiệu (tuần 2-3, xử lý hoàn toàn phía backend)

```
// lib/wheel.ts - quay o BACKEND, phía client chỉ nhận kết quả rồi chạy hoạt ảnh
const WHEEL = [
  { id: 'badge_bronze', weight: 30 },
  { id: 'badge_silver', weight: 20 },
  { id: 'badge_flame', weight: 15 },
  { id: 'discount_fee', weight: 10 }, // giảm phí giao dịch, không phải tiền mất
  // ... dư 10 o
];

export function spin(): string {
  const total = WHEEL.reduce((s, x) => s + x.weight, 0);
  let r = Math.random() * total;
  for (const item of WHEEL) {
    if ((r -= item.weight) <= 0) return item.id;
  }
  return WHEEL[WHEEL.length - 1].id;
}
```

Điểm dễ sai nhất ở phần này: nếu để việc quay số diễn ra ở phía trình duyệt, người dùng có thể tự sửa để chọn sẵn kết quả mong muốn. Kết quả phải luôn do backend quyết định và trả về, phía giao diện chỉ chạy hoạt ảnh cho khớp với kết quả đã có sẵn.

Giai đoạn 5: chương trình Anchor cho ống heo có kỳ hạn (tuần 3-4)

Đây là phần rủi ro kỹ thuật cao nhất của toàn bộ sản phẩm. Nguyên tắc nên theo: dựng thử một instruction gọi CPI đơn giản trước trong vài ngày đầu, chỉ cần gọi thành công vào Jupiter Earn một lần trên môi trường giả lập là đã xác nhận được hướng đi đúng, trước khi viết đầy đủ logic.

Cấu trúc tài khoản lưu trạng thái khóa

```
// programs/solstreak_vault/src/lib.rs
use anchor_lang::prelude::*;
use anchor_spl::token::{Token, TokenAccount};

declare_id!("YourProgramIdHere...");

#[account]
pub struct LockPosition {
    pub owner: Pubkey,
    pub asset_mint: Pubkey,
    pub principal: u64,
    pub ftoken_amount: u64,
    pub start_ts: i64,
    pub duration_secs: i64,
    pub tier_bps: u16,
    pub bump: u8,
}

#[derive(Accounts)]
pub struct LockDeposit<'info> {
    #[account(mut)]
    pub owner: Signer<'info>,
    #[account(
        init,
        payer = owner,
        space = 8 + 32 + 32 + 8 + 8 + 8 + 8 + 2 + 1,
        seeds = [b"lock", owner.key().as_ref(), &clock_seed()],
        bump
    )]
    pub position: Account<'info, LockPosition>,
    #[account(mut)]
    pub vault_ftoken_account: Account<'info, TokenAccount>,
    #[account(mut)]
    pub user_usdc_account: Account<'info, TokenAccount>,
    pub token_program: Program<'info, Token>,
    pub system_program: Program<'info, System>,
}
```

Hai instruction chính

```
pub fn lock_deposit(
    ctx: Context<LockDeposit>,
    amount: u64,
    duration_secs: i64,
    tier_bps: u16,
) -> Result<()> {
    require!(amount > 0, VaultError::ZeroAmount);
    require!(is_valid_tier(duration_secs, tier_bps), VaultError::InvalidTier);

    let ftoken_before = ctx.accounts.vault_ftoken_account.amount;
    jupiter_deposit_cpi(&ctx, amount)?;
    ctx.accounts.vault_ftoken_account.reload()?;
    let ftoken_received = ctx.accounts.vault_ftoken_account.amount - ftoken_before;

    let p = &mut ctx.accounts.position;
    p.owner = ctx.accounts.owner.key();
    p.principal = amount;
    p.ftoken_amount = ftoken_received;
    p.start_ts = Clock::get()?.unix_timestamp;
    p.duration_secs = duration_secs;
```

```

    p.tier_bps = tier_bps;
    Ok(())
}

pub fn unlock_withdraw(ctx: Context<UnlockWithdraw>) -> Result<()> {
    let p = &ctx.accounts.position;
    let now = Clock::get()?.unix_timestamp;
    let matured = now >= p.start_ts + p.duration_secs;

    let usdc_received = jupiter_withdraw_cpi(&ctx, p.ftoken_amount)?;

    let payout = if matured {
        let interest = (p.principal as u128)
            .checked_mul(p.tier_bps as u128).unwrap()
            .checked_mul(p.duration_secs as u128).unwrap()
            / 10_000 / (365 * 86_400);
        p.principal + interest as u64
    } else {
        p.principal
    };

    require!(usdc_received >= p.principal, VaultError::ProtocolShortfall);

    transfer_from_vault_to_user(&ctx, payout)?;
    transfer_from_vault_to_treasury(&ctx, usdc_received - payout)?;
    Ok(())
}

```

Những cái bẫy cụ thể của phần này

- Danh sách account cần cho CPI phải lấy đúng từ tài liệu chính thức của Jupiter, truyền vào qua `remaining_accounts`. Nếu Jupiter đổi cấu trúc account, chương trình sẽ hỏng, nên ghim đúng phiên bản SDK và có kiểm thử tích hợp chạy định kỳ.
- Lãi cam kết theo tier có thể cao hơn lãi thực tế Jupiter trả trong kỳ đó. Phần thiếu hụt cần được bù từ ngân sách dự phòng (treasury) của nền tảng, nếu không chuẩn bị trước, instruction rút tiền sẽ thất bại đúng lúc người dùng cần rút, đây là tình huống tệ nhất có thể xảy ra khi demo.
- Toàn bộ phép tính nhân chia liên quan đến lãi suất nên dùng kiểu số đủ lớn để tránh tràn số, và cần kiểm thử riêng trường hợp rút sớm một giây trước khi đủ hạn.
- Seed dùng để tạo tài khoản lưu vị trí khóa không nên chỉ dựa vào địa chỉ ví chủ sở hữu, nếu không người dùng sẽ không mở được cùng lúc hai ống heo, cần thêm một giá trị đếm hoặc mốc thời gian vào seed.
- Phần kiểm thử liên quan đến thời gian nên chạy trên môi trường giả lập cục bộ có thể tua thời gian; phần gọi CPI thật cần một môi trường giả lập lại trạng thái Mainnet vì Jupiter Lend không có Devnet dùng ổn định như đã nêu ở đầu tài liệu.
- Không nên triển khai lên Mainnet một chương trình chưa được kiểm toán mà lại nhận tiền thật của người khác. Trong giai đoạn thi, nên giới hạn số tiền tối đa được phép nạp cho mỗi vị trí khóa ngay trong chương trình, để giới hạn rủi ro nếu có lỗi chưa phát hiện.

Giai đoạn 6: ghép sản phẩm và chuẩn bị demo (tuần 4)

- Phần liên kết ngân hàng hoặc ví điện tử chỉ cần dựng giao diện minh họa, gắn nhãn rõ là tính năng sắp ra mắt, không cần hoạt động thật.
- Cần xác nhận lại chính xác tên và cách tích hợp của dịch vụ on-ramp mà cố vấn từng nhắc tới trước khi đưa vào slide chính thức, tránh nêu tên chưa chắc chắn trước giám khảo.
- Ghi rõ sản phẩm được xây dựng trên nền tảng Jupiter ngay trong giao diện và trong slide trình bày, đây là điều cố vấn đã nhấn mạnh, và giám khảo am hiểu DeFi chắc chắn sẽ hỏi đến.
- Chuẩn bị sẵn câu trả lời cho ba câu hỏi nhiều khả năng giám khảo sẽ đặt ra: nếu Jupiter gặp sự cố thì tiền người dùng ra sao, mức lãi cố định đã cam kết lấy nguồn nào bù khi lợi tức thị trường giảm, và sản phẩm nhận tiền tiết kiệm của người dùng Việt Nam có vướng vấn đề pháp lý nào không.

Tổng hợp: danh mục rủi ro xếp theo mức độ

Mức	Rủi ro	Giải pháp đã nêu ở
Red	Luồng ký giữa Privy và Jupiter không chạy được	Giai đoạn 2, nên làm đầu tiên, thử nghiệm nhanh trong 2-3 ngày
Red	Gọi CPI vào Jupiter Lend sai cấu trúc account hoặc dữ liệu	Giai đoạn 5, thử CPI đơn giản trên môi trường giả lập trước khi viết đầy đủ
Red	Lãi cam kết cao hơn lợi tức thực tế, thiếu tiền trả	Bù từ ngân sách dự phòng của nền tảng, ghi rõ nhãn lãi ước tính
Yellow	Không chắc test được trên Devnet	Môi trường giả lập trạng thái Mainnet cục bộ, cộng với tiền thật số nhỏ khi cần
Yellow	Ví mới không có SOL để trả phí	Cấp một khoản SOL nhỏ cho ví mới trong giai đoạn demo
Yellow	Tính chuỗi sai do mạng nghẽn hoặc lệch múi giờ	Vùng đệm ba giờ, tính ngày theo giờ Việt Nam, luôn xác minh lại trên chain
Yellow	Người dùng gian lận kết quả chuỗi hoặc vòng quay	Mọi việc xác minh và random đều xử lý ở phía backend
Green	Giới hạn tốc độ RPC, hết hạn blockhash, nhầm đơn vị thập phân	Bảng chi tiết ở giai đoạn 2

Lộ trình bốn tuần này đi theo đúng thứ tự ba cửa kỹ thuật: Privy, Jupiter Earn, rồi mới đến chương trình khóa kỳ hạn. Điểm cần khắc cốt: tuần đầu tiên chưa cần giao diện đẹp, chỉ cần một trang đơn giản có nút đăng nhập, nạp một USDC, rút ra, chạy tron tru từ đầu đến cuối. Toàn bộ phần còn lại của sản phẩm, gồm chuỗi ngày, vòng quay, ổng heo có kỳ hạn, đều đứng trên nền tảng đó.